DOCUMENTATION TECHNIQUE

WestPay *API* Reference

Intégrez les paiements Mobile M
Centre dans votre application v
WestPay.

v2.0 — Stable

REST / JS

westpay.cloud

- Paiements Mobile Money USSD push
- Retraits automatiques temps réel

01 — INTRODUCTION

Vue d'ensemble

WestPay est une plateforme d'agrégation de paiements Mobile Money en Afrique de l'Ouest et du Centre. L'API REST permet d'intégrer les paiements, les retraits automatiques, la consultation des soldes et les notifications en temps réel dans n'importe quelle application web ou mobile.

URL DE BASE	FORMAT
`https://westpay.cloud`	`application/json`
MÉTHODES HTTP	**AUTHENTIFICATION**
GET · POST · PUT · PATCH · DELETE	JWT Bearer + Clé API

🔐 **SÉCURITÉ — IMPORTANT**

Ne partagez **jamais** vos clés API ni votre secret webhook. Régénérez-les immédiatement si vous suspectez une compromission.

FONCTIONNALITÉS PRINCIPALES

- ✓ Paiements Mobile Money via USSD push
- ✓ Retraits automatiques vers Mobile Money
- ✓ Notifications webhook en temps réel
- ✓ Support multi-pays et multi-opérateurs
- ✓ Page de paiement sécurisée et hébergée (RobotPay)
- ✓ Consultation des soldes par pays
- ✓ Historique complet des transactions
- ✓ Signature HMAC-SHA256 pour la sécurité

02 — AUTHENTIFICATION

Authentification

L'API utilise deux niveaux d'authentification : un token JWT obtenu après connexion, et une clé API par pays disponible dans votre tableau de bord.

1 OBTENIR UN TOKEN JWT

POST `/api/auth/merchant/login` — Connexion marchand

Envoie vos identifiants pour obtenir un token JWT valable 24h.

JSON — REQUEST BODY

```
{
  "email": "contact@votreentreprise.com",
  "password": "votre_mot_de_passe"
}
```

JSON — RESPONSE 200

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

2 INCLURE LE TOKEN DANS CHAQUE REQUÊTE

Ajoutez ces headers à toutes vos requêtes authentifiées :

```
Authorization: Bearer <VOTRE_JWT_TOKEN>
Content-Type: application/json
```

> ℹ️ **EXPIRATION DU TOKEN**

> Le token JWT expire après **24h**. Prévoyez un mécanisme de re-connexion automatique dans votre code.

3 CLÉ API PAR PAYS

Chaque pays actif dispose d'une clé API unique. Disponible dans votre dashboard sous **"Clés API"**. Format : `PREFIX-[40 caractères]`

```
X-API-KEY: TGO-a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0
```

Ce header est optionnel mais requis pour certains endpoints (consultation par pays, retraits).

---

03 — ENDPOINTS

Endpoints — Consultation

Endpoints de lecture pour récupérer vos soldes, transactions, statistiques et clés API.

GET	/api/merchant/balance	Soldes par pays
GET	/api/merchant/transactions	Historique des transactions
GET	/api/merchant/stats	Statistiques du compte
GET	/api/merchant/api-keys	Liste des clés API actives
POST	/api/merchant/regenerate-api	Régénérer une clé API

Paiements (Dépôt)

WestPay fournit une page de paiement hébergée et sécurisée. Redirigez simplement vos utilisateurs vers cette URL — ils entrent leur numéro Mobile Money et valident le paiement directement sur leur téléphone via USSD.

Page de paiement hébergée — RobotPay

Redirigez votre utilisateur vers l'URL de paiement :

```
https://westpay.cloud/pay?merchant=votre-slug&amount=5000&country=Togo&redirect=https%3A%2F%2Fvotresite.com%2Fmerci
```

1. Redirigez votre utilisateur vers l'URL de paiement WestPay.
2. Le client entre son numéro et valide le paiement USSD sur son téléphone.
3. Votre URL de redirection reçoit : `?status=success&amount=5000&ref=OP-abc123`

Paramètres de l'URL de paiement

PARAMÈTRE	REQUIS	DESCRIPTION
`merchant`	Requis	Votre slug marchand (ex: `ecomat`)
`amount`	Requis	Montant en F CFA (entier positif)
`country`	Optionnel	Pays (ex: `Togo`). Premier pays actif si omis
`redirect`	Optionnel	URL de retour après paiement (encoder avec `encodeURIComponent`)

Opérateurs disponibles par pays

🇹🇬 **Togo** +228

🇧🇯 **Bénin** +229

🇧🇫 **Burkina Faso** +226

Moov Money, TMoney

MTN Mobile Money

Moov Money, Orange Money

🇨🇮 **Côte d'Ivoire** +225

Moov Money, MTN, Orange Money, Wave

🇸🇳 **Sénégal** +221

Mixx by Yas, Orange Money, Wave

🇲🇱 **Mali** +223

Orange Money

🇨🇲 **Cameroun** +237

MTN Mobile Money, Orange Money

🇨🇬 **Congo Brazzaville** +242

MTN Mobile Money

🇬🇦 **Gabon** +241

Airtel Money, Moov Money

⚠️ ORANGE MONEY — BURKINA FASO : CODE OTP OBLIGATOIRE

Pour payer via Orange Money au Burkina Faso, le client doit d'abord générer un code OTP depuis son téléphone, puis le saisir sur la page de paiement.

Composer sur le téléphone : `*144*6*montant#`
Ex. pour 5 000 F CFA : `*144*6*5000#`

Un champ de saisie OTP s'affiche automatiquement sur la page de paiement RobotPay lorsque Orange Money Burkina Faso est sélectionné.

05 — RETRAITS

Retraits automatiques (Transferts)

Envoyez de l'argent directement vers n'importe quel portefeuille Mobile Money. Le montant est débité de votre solde et transféré instantanément au destinataire.

Flux de retrait

1. Appelez l'endpoint avec le numéro, le montant et le nom du destinataire.
2. WestPay vérifie votre solde disponible.
3. Le montant est transféré sur le portefeuille Mobile Money du destinataire.
4. Votre solde est débité (montant + frais éventuels).
5. La transaction est enregistrée avec un `txId` préfixé `TR-`.

> ⚠️ **FORMAT DU NUMÉRO DE TÉLÉPHONE (MSISDN)**
>
> Le numéro doit toujours inclure l'indicatif pays, sans le `+`.

EXEMPLES MSISDN

```
// Togo
228 + 90123456    → 22890123456
// Côte d'Ivoire
225 + 0789012345 → 2250789012345
// Bénin
229 + 97123456    → 22997123456
// Sénégal
221 + 771234567   → 221771234567
```

`POST` `/api/merchant/transfer` Lancer un retrait

CHAMP	TYPE	REQUIS	DESCRIPTION
`country`	string	Requis	Nom du pays (ex: `Togo`)
`msisdn`	string	Requis	Numéro avec indicatif pays, sans `+`
`amount`	integer	Requis	Montant en F CFA
`firstName`	string	Requis	Prénom du destinataire
`lastName`	string	Requis	Nom du destinataire

JSON — RESPONSE 200

```
{
  "reference": "TR-abc123def456",
  "status":    "success",
  "fees":      150
}
```

06 — WEBHOOKS

Webhooks & Notifications

Configurez une URL webhook pour recevoir des notifications en temps réel à chaque paiement confirmé. WestPay envoie un POST signé avec votre secret via HMAC-SHA256.

Configuration en 4 étapes

1. Allez dans votre dashboard, onglet **"Webhook"**.
2. Entrez votre URL (ex : `https://votresite.com/api/webhook`).
3. Copiez et sauvegardez votre **secret webhook**.
4. Cliquez **"Tester"** pour valider la configuration.

Payload reçu sur votre endpoint

HTTP POST — WEBHOOK PAYLOAD

```
// Headers envoyés par WestPay
X-RobotPay-Signature: a3f8c2d1e4b7...  (HMAC-SHA256)
X-RobotPay-Event:     payment.confirmed
Content-Type:         application/json

// Body JSON
{
  "event":        "payment.confirmed",
  "txId":         "OP-abc123def456",
  "amount":       5000,
  "currency":     "XOF",
  "payer":        "+22890123456",
  "country":      "Togo",
  "merchantSlug": "votre-slug",
  "provider":     "mobile",
  "timestamp":    "2026-03-01T10:30:00.000Z"
}
```

Endpoints Webhook

GET	/api/merchant/webhook	Récupérer la configuration webhook
PUT	/api/merchant/webhook	Mettre à jour l'URL webhook
POST	/api/merchant/webhook/test	Envoyer un webhook de test
GET	/api/merchant/webhook/logs	Consulter les logs webhook

07 — EXEMPLES DE CODE

Exemples de code

1 — Connexion + Récupération du solde

JAVASCRIPT — NODE.JS / BROWSER

```javascript
const BASE = "https://westpay.cloud";

// 1. Connexion
const loginRes = await fetch(`${BASE}/api/auth/merchant/login`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    email:    "contact@votreentreprise.com",
    password: "votre_mot_de_passe"
  })
});
const { token } = await loginRes.json();

// 2. Récupérer le solde
const balanceRes = await fetch(`${BASE}/api/merchant/balance`, {
  headers: { "Authorization": `Bearer ${token}` }
});
const soldes = await balanceRes.json();
console.log("Soldes:", soldes);
// [{ country: "Togo", balance: 125000 }, ...]
```

2 — Rediriger vers la page de paiement

JAVASCRIPT

```javascript
const BASE = "https://westpay.cloud";

// Construire l'URL de paiement et rediriger le client
const url = new URL(`${BASE}/pay`);
url.searchParams.set("merchant", "votre-slug");
url.searchParams.set("amount",   "5000");
url.searchParams.set("country",  "Togo");
url.searchParams.set("redirect", "https://votresite.com/merci");

window.location.href = url.toString();

// Après paiement, le client est redirigé vers :
// https://votresite.com/merci?status=success&amount=5000&ref=OP-abc123
```

3 — Effectuer un retrait (transfert)

JAVASCRIPT

```javascript
const BASE = "https://westpay.cloud";

async function effectuerRetrait(token, data) {
  const res = await fetch(`${BASE}/api/merchant/transfer`, {
    method: "POST",
    headers: {
      "Content-Type":  "application/json",
      "Authorization": `Bearer ${token}`
    },
    body: JSON.stringify(data)
  });

  if (!res.ok) {
    const err = await res.json();
    throw new Error(err.message || "Erreur retrait");
  }
  return res.json();
}

// Utilisation
try {
  const resultat = await effectuerRetrait(token, {
    country:   "Togo",
    msisdn:    "22890123456",
    amount:    5000,
    firstName: "Marie",
    lastName:  "Konan"
  });
  console.log("Retrait réussi !");
  console.log("Référence:", resultat.reference);
  console.log("Frais:", resultat.fees, "F CFA");
} catch (err) {
  console.error("Erreur:", err.message);
}
```

4 — Recevoir et vérifier un webhook

JAVASCRIPT — EXPRESS.JS

```javascript
const express = require("express");
const crypto  = require("crypto");
const app     = express();
app.use(express.json());

const WEBHOOK_SECRET = process.env.WESTPAY_WEBHOOK_SECRET;

function verifierSignature(body, signature) {
  const attendu = crypto
    .createHmac("sha256", WEBHOOK_SECRET)
    .update(JSON.stringify(body))
    .digest("hex");
  return crypto.timingSafeEqual(
    Buffer.from(signature),
    Buffer.from(attendu)
  );
}

app.post("/api/webhook", (req, res) => {
  const signature = req.headers["x-robotpay-signature"];
  const event     = req.headers["x-robotpay-event"];

  if (!signature || !verifierSignature(req.body, signature)) {
    return res.status(401).json({ error: "Signature invalide" });
  }

  if (event === "payment.confirmed") {
    const { txId, amount, payer, country } = req.body;
    console.log(`Paiement confirmé: ${amount} F CFA de ${payer} (${country})`);
    // Mettre à jour votre base de données ici
    // await db.marquerPaiementConfirme(txId, amount);
  }

  res.json({ received: true }); // Répondre 200 rapidement
});
```

08 — RÉFÉRENCE

Gestion des erreurs

L'API retourne toujours un objet JSON avec un champ `message` en cas d'erreur.

Format d'erreur

JSON — ERROR RESPONSE

```
{
  "message": "Description de l'erreur",
  "code":    "ERROR_CODE"  // optionnel
}
```

Codes HTTP

CODE	CAUSE
200	Succès
400	Requête invalide
400	Solde insuffisant
401	Non autorisé
401	Signature invalide
403	Accès refusé
404	Ressource introuvable
409	Conflit
429	Trop de requêtes
500	Erreur serveur

Gestion d'erreur robuste

JAVASCRIPT — ERROR HANDLER

```javascript
async function appelAPI(url, options = {}) {
  try {
    const res = await fetch(url, {
      ...options,
      headers: {
        "Content-Type":  "application/json",
        "Authorization": `Bearer ${token}`,
        ...(options.headers || {})
      }
    });

    const data = await res.json();

    if (!res.ok) {
      switch (res.status) {
        case 401:
          await seReconnecter();
          throw new Error("Session expirée, veuillez vous reconnecter");
        case 400:
          throw new Error(data.message || "Requête invalide");
        case 429:
          await new Promise(r => setTimeout(r, 5000));
          return appelAPI(url, options); // retry
        default:
          throw new Error(data.message || `Erreur ${res.status}`);
      }
    }

    return data;
  } catch (err) {
    console.error("Erreur API:", err.message);
    throw err;
  }
}
```

Bonnes pratiques de sécurité

🔑 Protéger la clé API et le token JWT

- ✓ Ne jamais inclure la clé API directement dans le code source versionné (GitHub, etc.)
- ✓ Utiliser des variables d'environnement : `process.env.WESTPAY_API_KEY` / `os.getenv('WESTPAY_API_KEY')`
- ✓ Régénérer la clé API immédiatement si vous suspectez une compromission
- ✓ Le token JWT expire après 24h — prévoir un mécanisme de re-connexion automatique

🔒 Sécuriser votre endpoint webhook

- ✓ Toujours vérifier la signature HMAC-SHA256 avant de traiter un webhook
- ✓ Utiliser `crypto.timingSafeEqual` (JS) ou `hash_equals` (PHP) pour éviter les timing attacks
- ✓ Répondre 200 rapidement, puis traiter en arrière-plan (éviter les timeouts)
- ✓ Enregistrer tous les webhooks reçus dans une file d'attente (queue) pour le traitement

✅ Valider les données reçues

- ✓ Toujours vérifier que le montant dans le webhook correspond au montant attendu
- ✓ Vérifier que le `merchantSlug` correspond bien à votre compte
- ✓ Implémenter une protection contre la rejouabilité (stocker les `txId` traités)
- ✓ Valider le format des numéros de téléphone avant de lancer un retrait

⚙️ Pratiques de code

- ✓ Utiliser HTTPS uniquement pour votre URL webhook (HTTP sera refusé)
- ✓ Implémenter un mécanisme de retry avec backoff exponentiel pour les erreurs 5xx
- ✓ Limiter les appels API avec du rate limiting côté client
- ✓ Logger les appels API avec l'ID de transaction pour faciliter le support

Exemple — Variables d'environnement

.ENV + NODE.JS

```
# Fichier .env (NE PAS committer ce fichier !)
WESTPAY_API_KEY=TGO-votre_cle_api_complete
WESTPAY_WEBHOOK_SECRET=votre_secret_webhook
WESTPAY_MERCHANT_EMAIL=contact@votreentreprise.com
WESTPAY_MERCHANT_PASSWORD=votre_mot_de_passe

// Dans votre code Node.js :
require("dotenv").config();

const API_KEY        = process.env.WESTPAY_API_KEY;
const WEBHOOK_SECRET = process.env.WESTPAY_WEBHOOK_SECRET;

// Fichier .gitignore :
// .env
// node_modules/
```

💡 **SUPPORT**

Pour toute question technique, contactez votre gestionnaire de compte WestPay.

© 2026 WestPay · Usage restreint